NeuroRoute

A rule-first AI model router: cost-, latency- and quality-aware

Problem A-01 (ModelRouter) | Domain 2: AI Systems and Infrastructure | VORTEXA 3.0 | Build time: 6 hours

In one line: NeuroRoute picks the cheapest AI model that is guaranteed to follow every rule of the request, and it proves it by showing why every other model was rejected.

1. Problem Understanding

The picture for a non-technical reader. Think of a hospital with a nurse (cheap, fast, handles small problems), a physician (medium) and a surgeon (expensive, slow, handles serious cases). Sending every patient to the surgeon wastes money. Sending every patient to the nurse harms the serious cases. And if the doctor you chose does not show up, the backup must still be right for that patient, not just whoever is free.

Replace the doctors with **AI language models**. Companies can use several, and they differ in price, speed, how smart they are, how much text they can read at once, and how often they fail.

What goes wrong today	Why it hurts
Every request goes to the strongest model	Money is wasted on easy questions
Every request goes to the cheapest model	Hard tasks (e.g. a long contract) get poor answers
The app is hard-wired to one model	If that model is slow or down, the feature breaks
A backup is chosen in a hurry	It can silently break a rule the first choice respected (too slow, too costly, too small)

The four hard limits that every request carries. "Hard" means no exceptions: a model that breaks even one is not a valid choice.

Limit	Meaning	Example
max_latency_ms	Answer must arrive within this time	2000 ms
max_cost	Request must not cost more than this	0.10 units
min_capability	Model must be at least this capable (level 1 to 5)	level 3
required_context	Model must be able to read this much text (tokens)	20,000 tokens

What the problem statement requires us to deliver:

- Choose among *valid* models by a stated preference (for example lowest cost) and explain every decision: which models were considered and why others were rejected.
- Treat endpoints as unreliable: on failure or slowness, fail over to another model that still meets *every* limit. Failover must never relax a limit.
- If no model fits, say so clearly instead of sending the request to an invalid model.
- Treat any complexity estimate from the text as uncertain; when unsure, choose the safer option.
- Track cost, latency, failures and decisions. Judges must be able to change a model's latency or availability and see a valid alternative chosen.

2. Proposed Solution

We build an **inference gateway**: one door that every request passes through. It is a small web service, not a network proxy. Behind the door sits a decision engine with one guiding principle: **rules first, AI second**.

Step	What happens	How
1. Validate	Reject malformed requests at the door	FastAPI + Pydantic
2. Estimate	Count tokens, estimate cost per model	Length / 4 (plain code)
3. Classify (AI)	Guess how hard the prompt is, with a confidence	scikit-learn classifier
4. Hard filter	Remove every model that breaks any limit; record the reason	Plain Python, unit-tested
5. Rank	Order the valid models by the stated preference	Sort by cost, speed or capability
6. Call	Call the top model; timeout equals the time budget	asyncio timeout
7. Failover	On error or timeout, take the next model from the same valid list; re-check remaining time	No re-filtering with looser limits
8. Refuse	If the valid list is empty or used up, return NO_VALID_MODEL with reasons	Nothing is sent to an invalid model
9. Record	Store cost, latency, failures, decision trace	SQLite + live dashboard

Example decision trace (what judges see for every request):

Model	Valid?	Reason / detail
nano-1	No	capability 1 < 3
mini-2	No	capability 2 < 3
pro-3	Yes	est. cost 0.06, latency 1500 ms (chosen)
ultra-4	No	latency 3000 > 2000
max-5	No	latency 5500 > 2000

How judges test it live: a dashboard lets them switch a model off, raise its latency or set a failure rate, then send a request and watch a different *valid* model answer. Impossible limits (for example 100 ms with capability 5) return a clear refusal. Everything runs offline on fictional, simulated models, so results are reproducible.

3. Intended Users

User	Their need	What NeuroRoute gives them
App and backend developers	Use several AI models without hard-coding one; survive outages	One endpoint; automatic safe failover; clear errors
Product and cost owners	Control AI spend without hurting quality	Cheapest valid model per request; per-request cost log
Platform and reliability teams	Review behaviour, find slow or failing models	Decision traces, failure counts, latency history
Hackathon judges and reviewers	Verify the rules hold under change	Live control panel: change latency or availability and see a valid alternative

Assumption: users send requests that already carry explicit limits, as the problem statement defines. Non-technical stakeholders are served by the dashboard and the plain-language decision trace.

4. AI Integration

The hackathon requires AI to be used meaningfully and requires decisions with real consequences to follow explicit, checkable rules. NeuroRoute therefore splits the work cleanly:

	Rule-based (deterministic)	AI (advisory)
Role	Decides which models are **allowed**	Estimates how **hard** the prompt is
Technique	Plain Python checks on latency, cost, capability, context, health	TF-IDF text features + Logistic Regression (scikit-learn)
Output	Valid list and rejection reasons	Complexity level (1 to 5) and a confidence score
Can it override the other?	Yes. Rules always win.	No. It may only **raise** the needed capability.

Why a classifier? Requests often do not state how hard they are. Reading the text (words like "analyse", "clause", "compare", length, code, numbers) lets the router avoid sending a hard task to a weak model, which is a language-understanding task a rule list cannot cover well.

How uncertainty is handled (a direct requirement of the problem):

- The classifier returns a confidence. If it is below a threshold (starting value 0.6, to be tuned on our sample prompts), the router **assumes the harder level**.
- The AI estimate can only raise the needed capability. The user's own minimum is never lowered.
- If raising the level leaves no valid model, the router falls back to the user's stated minimum and flags the trace as *low-confidence routing*. Cost and latency limits are never broken.
- Training data: about 60 hand-written fictional prompts labelled by level. Small, CPU-only, runs offline, no paid API.
- If the classifier is unavailable, the router still works on the rule-based core alone.

5. Differentiation from Existing Solutions

Good tools already exist for calling many models, saving cost with a difficulty classifier, and logging. We reuse their ideas. What we could not find as a core feature is a **constraint-first, explainable, never-relax-on-failover** decision per request. This comparison is based on our reading of how these tools are generally described and should be re-checked before the final presentation.

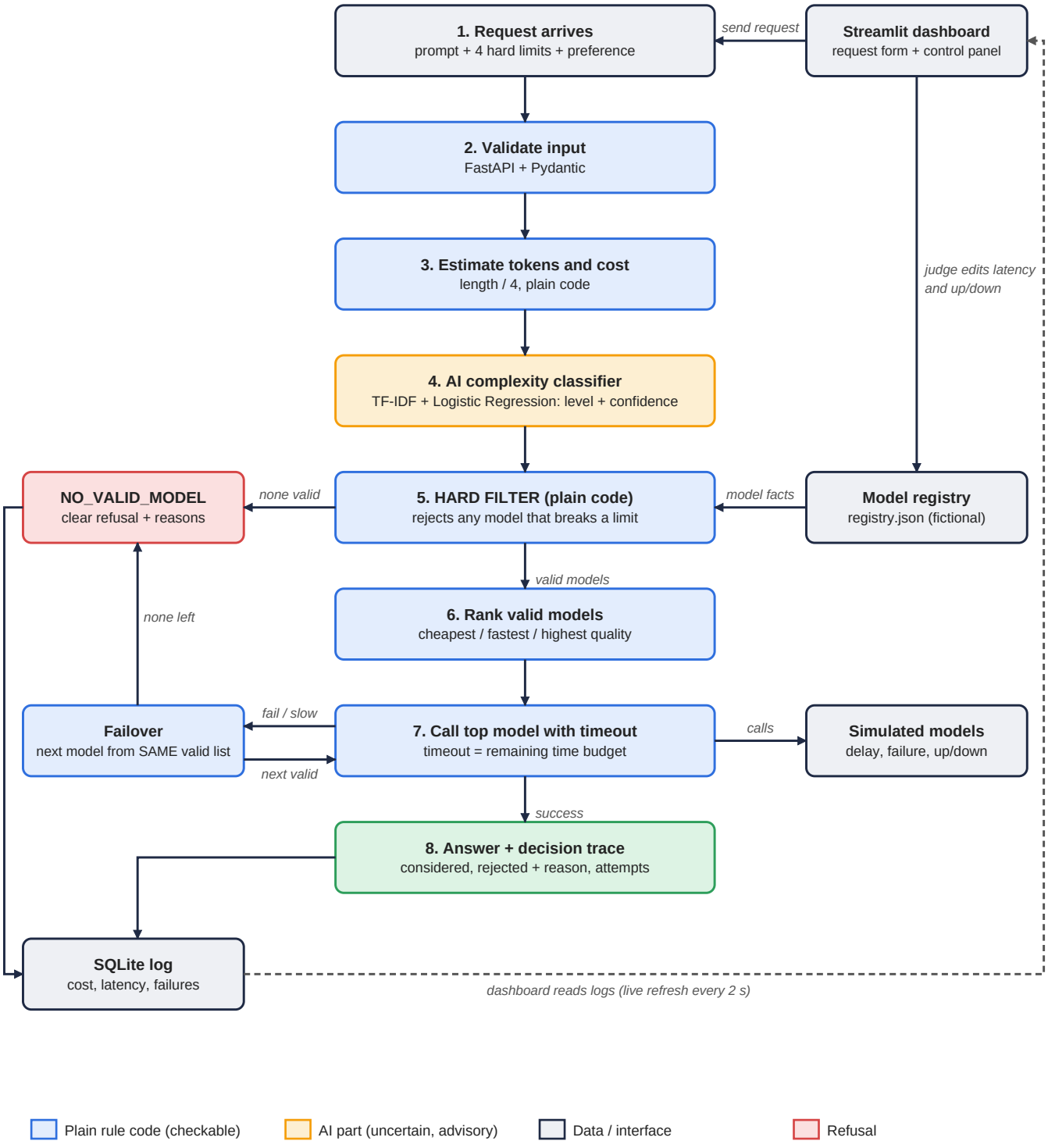
Existing tool	What it does well	Gap against this problem	What we borrow
LiteLLM	One interface to many models; fallbacks, retries, cost tracking	Per-request hard limits with a rejection list are not its main focus	Idea of fallback lists and cost tracking
OpenRouter	Hosted access to many models	Paid and external; decision logic is not ours to show	Reference only
RouteLLM	Classifier chooses a strong or weak model to save cost	Two-model choice; no hard limits or safe failover	Classifier-driven routing idea
Martian / Not Diamond	Commercial predictive routers	Paid and closed	Reference only
Portkey / Helicone / Cloudflare AI Gateway	Logging, retries, dashboards	Hosted; not constraint-first	Logging and dashboard idea
Ollama	Runs small models locally	Not a router	Optional local model, only if time remains

Our differentiators

Differentiator	Meaning
Guarantee, not optimisation	A model that breaks any hard limit can never be chosen, including during failover
Proof for every decision	Each response lists considered and rejected models with reasons
AI that admits doubt	Low confidence leads to the safer option, shown in the trace
Honest refusal	NO_VALID_MODEL instead of a risky guess
Live break-it test	Judges change latency or availability and watch a valid alternative appear
Fully offline	Simulated models and a local classifier; no paid API or GPU

6. System Architecture

The centre column is the request pipeline. Orange marks the only AI step. Blue steps are plain, checkable code. Failover loops back to the call step using only models that already passed the hard filter.



Rule: the AI estimate (step 4) may only raise the needed capability. It never lowers the user's own minimum.

Tech stack (locked): Python 3.10+, FastAPI, Pydantic, scikit-learn, asyncio, SQLite, Streamlit, pytest.

Scope Boundary (locked, to avoid scope creep)

We build	We do not build
API gateway with POST /route; hard filter; ranking; safe failover; clear refusal; decision trace	Browser extension; desktop app; login or billing; real network proxy
AI complexity classifier with confidence and safer-option rule	LLM training or fine-tuning; embeddings (only if time remains)
Simulated models with adjustable delay, failure rate, up/down	Paid APIs or real customer data; LiteLLM or OpenRouter in the core path
SQLite logging and a Streamlit dashboard with live refresh and judge controls	Streaming, caching, rate limiting, multi-cloud deployment
pytest tests for filter, ranking, failover, refusal	Static slides or mock-up-only UI as the deliverable

If time runs short, cut in this order: charts, then the highest-quality preference, then (last resort) the classifier. Never cut the hard filter, the safe failover, the refusal, or the decision trace.